

14-Day Roadmap: RAG Project + Full AI/ML Syllabus

Days 1-6: build a production-quality RAG app. Days 7-14: cover the full Skillgence AI/ML syllabus (the classical AI, ML, RL, and deep learning content that doesn't naturally fit inside a RAG project).

How This Plan Is Structured

This extends the original 6-day RAG roadmap rather than replacing it. RAG stays the priority and comes first — it's still the project you'll demo and defend. Days 7-14 exist because the internship syllabus covers a lot of classical AI/ML/DL/RL/CV content that genuinely has no honest connection to a text-based RAG app — search algorithms, game playing, image classification, reinforcement learning, etc. Rather than force fake ties between those topics and RAG, this plan is upfront: some days build toward RAG, some days build toward passing/participating in the broader syllabus, and each section says clearly which one it is.

The syllabus itself is listed as a 40-hour certificate course. Days 1-6 run roughly 24-30 hours; Days 7-14 (8 days x ~5-6 hrs, since you said intensive/longer days) covers close to the full remaining 40-hour scope. That's not a coincidence — it's sized to match what the syllabus actually expects.

Legend Used Throughout Days 7-14

- Link back to RAG — a genuine, non-forced connection to the project you already built.
- Honest note — flags where a topic has no real connection to RAG. You're learning it because it's on the syllabus, not because it improves your project.

PART 1 — Days 1-6: RAG Project

This part is unchanged from the original roadmap. Full day-by-day detail (code snippets, exact library calls, checkpoints) is in the previously generated RAG_6Day_Roadmap document — summarized here so this doc stands on its own.

Day 1 — Python Environment, Just-Enough AI/ML Fundamentals, Project Skeleton

- Virtual environment, project folder structure, Git repo pushed to GitHub on Day 1 (not Day 6).
- Minimum AI/ML theory needed to understand RAG: what an embedding is, what an LLM does, why RAG exists.
- Install LangChain, ChromaDB, sentence-transformers, PyMuPDF, Streamlit. Get a free Gemini API key, verify it works.

Checkpoint: Repo live, venv working, Gemini API call returns a response.

Day 2 — Data Handling and PDF Text Extraction

- Just-enough NumPy/Pandas: DataFrame basics, array math — not a full course.
- PyMuPDF text extraction from PDFs, tested on clean/scanned/table-heavy PDFs.
- Multi-document loader that tags every extracted page with its source filename.

| **Checkpoint:** A folder of PDFs → structured list of dicts with text, source, and page number.

Day 3 — NLP Basics, Chunking, Embeddings, Vector Database

- Tokenization basics. Chunking strategy with RecursiveCharacterTextSplitter, tested at multiple chunk sizes.
- Embeddings via sentence-transformers (all-MiniLM-L6-v2). Manual cosine similarity in NumPy.
- ChromaDB: store (vector, text, metadata), query by similarity. Skim FAISS's API for comparison.

| **Checkpoint:** A PDF → chunked, embedded, stored in ChromaDB, queryable with a test question.

Day 4 — LLMs, Prompt Engineering, LangChain, Retrieval Pipeline

- Prompt engineering: instruct the model to answer only from context, say "I don't know" otherwise. Low temperature for factual answers.
- LangChain glue: Document objects, Retriever interface, ChatGoogleGenerativeAI for Gemini.
- Retrieval + generation functions wired together, tested end to end manually.

| **Checkpoint:** A function that takes a question and returns a grounded answer from retrieved chunks.

Day 5 — Full RAG Pipeline: Multi-Doc, Memory, Errors, Evaluation

- Assembled pipeline: ingest → chunk → embed → store → retrieve → generate, with source citations.
- Multi-document support (tested with 2 unrelated PDFs). Simple list-based conversation memory.
- Error handling: corrupted PDFs, empty index, API failures, zero-relevance queries.
- A 5-10 question evaluation set with known answers, run through the pipeline and checked manually.

| **Checkpoint:** Full pipeline works end-to-end from the terminal, with an evaluation set and results.

Day 6 — Streamlit UI, GitHub Polish, Deployment, Interview Prep

- Streamlit frontend: PDF upload, question box, source citations, chat history via session_state.
- Clean requirements.txt, .gitignore, and a real README (what it does, architecture, setup, screenshot, limitations).
- Optional deployment to Streamlit Community Cloud.
- Mock 5-minute presentation + prepared answers for likely interview questions (why RAG over fine-tuning, chunk size trade-offs, scaling, evaluation, limitations).

| **Checkpoint:** Working Streamlit app, clean GitHub repo, deployed link (optional), and you can explain every part of the architecture unaided.

PART 2 — Days 7-14: Full AI/ML Syllabus Coverage

This part exists purely to cover what the syllabus requires and what RAG doesn't. Expect a mix of theory, small standalone code exercises, and two syllabus-mandated mini-projects (image classification, object detection). These are separate from your RAG app — don't try to merge them artificially.

Day 7 — Intelligent Agents and Search Algorithms

Syllabus topics: Introduction to Intelligent Agents, Problem Formulation, State Space Formulation, Uninformed Search Strategies, Informed Search Strategies, Iterative Deepening, Heuristics, Constraint Satisfaction Problems.

What to actually learn

- An intelligent agent: perceives an environment through sensors and acts on it through actuators to achieve a goal.
- Problem formulation: define initial state, actions, transition model, goal test, path cost — the standard way any search problem is framed.
- Uninformed search: BFS and DFS — implement both on a simple graph or maze in Python.
- Informed search: A* and greedy best-first search, using a heuristic (e.g. Manhattan distance for grid mazes).
- Iterative deepening: repeated depth-limited DFS with increasing depth limits — combines DFS's low memory use with BFS's completeness.
- Constraint Satisfaction Problems (CSPs): variables, domains, constraints — classic example is Sudoku or map coloring. Implement basic backtracking search.

Hands-on

- Build a maze solver using BFS, DFS, and A* — compare path length and nodes explored between them.
- Implement a simple CSP solver (map coloring with backtracking) — 30-40 lines is enough to demonstrate understanding.

Honest note: Search algorithms are foundational symbolic AI, not used anywhere in RAG. RAG's "search" (vector similarity) is a completely different mechanism — don't conflate the two if asked to explain either.

Checkpoint: A working maze solver comparing BFS/DFS/A*, and a basic CSP backtracking solver.

Day 8 — Game Playing, Expert Systems, Inductive Learning, NLP Basics

Syllabus topics: Game Playing, Expert Systems, Natural Language Processing, Inductive Learning.

What to actually learn

- Game playing: minimax algorithm for two-player zero-sum games, and alpha-beta pruning to cut unnecessary branches. Implement minimax for Tic-Tac-Toe — small enough to fully trace by hand.
- Expert systems: rule-based reasoning (if-then rules + an inference engine). Build a trivial rule-based system (e.g. a 10-rule diagnostic "expert" for a toy domain) to see the pattern — this is the ancestor of modern LLM-based reasoning, worth knowing as historical/conceptual context.
- Inductive learning: learning a general rule from specific examples — the core idea behind all of supervised ML (Day 9). Understanding this now makes Day 9 click faster.
- NLP basics (classical, pre-transformer): bag-of-words, TF-IDF, stemming/lemmatization, POS tagging. This is the "old way" of representing text numerically, before embeddings existed.

Hands-on

- Implement minimax + alpha-beta pruning for Tic-Tac-Toe, playable against you from the terminal.
- Build a TF-IDF vectorizer on a handful of sentences using scikit-learn, and compare document similarity with TF-IDF vs. the sentence-transformer embeddings you used in RAG.

Link back to RAG: The TF-IDF vs. embeddings comparison is a genuinely great thing to know cold — it's very likely you'll be asked "why embeddings and not just keyword matching / TF-IDF?" in an interview about your RAG project. Now you'll have hands-on proof of the difference, not just a memorized answer.

Checkpoint: A playable Tic-Tac-Toe AI using minimax, and a side-by-side TF-IDF vs. embedding similarity comparison.

Day 9 — Supervised Learning

Syllabus topics: KNN, SVM Classifier, Naive Bayes Classifier, Regression Learning, Logistic Regression, Decision Trees, Boosting and Bagging.

What to actually learn

- The supervised learning setup: labeled training data (X, y), a model that learns a mapping, evaluated on held-out test data.
- K-Nearest Neighbors: classify a point by majority vote of its k closest neighbors — mechanically similar to how vector similarity search works in RAG, but applied to labeled classification instead of retrieval.
- Naive Bayes: probabilistic classifier assuming feature independence — commonly used historically for spam/text classification.
- Linear and logistic regression: regression predicts a continuous value, logistic regression predicts a class probability via the sigmoid function.
- Decision Trees: recursively split data by the feature that best separates classes (e.g. by information gain / Gini impurity).
- SVM: finds the maximum-margin hyperplane separating classes.
- Boosting and bagging: ensemble methods — bagging trains many models on random subsets and averages them (Random Forest); boosting trains models sequentially, each correcting the previous one's errors (e.g. XGBoost, AdaBoost).

Hands-on — the syllabus's own suggested project

- Data Prediction using different ML algorithms: pick one clean tabular dataset (e.g. from scikit-learn's built-in datasets or Kaggle), and run KNN, Naive Bayes, Logistic Regression, Decision Tree, and a Random Forest on it. Compare accuracy.

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

models = {
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
}

for name, model in models.items():
    model.fit(X_train, y_train)
```

```
preds = model.predict(X_test)
print(name, accuracy_score(y_test, preds))
```

Honest note: None of these classical models are used inside your RAG pipeline — RAG doesn't classify anything, it retrieves and generates. This day exists to satisfy the syllabus's own listed "Data Prediction" project requirement, independent of RAG.

Checkpoint: Five classical ML models trained and compared on one dataset, with accuracy scores.

Day 10 — Unsupervised Learning, Factor Analysis, Data Visualization

Syllabus topics: Clustering, K-Means, EM Algorithm, Mixture of Gaussians, PCA, ICA, Latent Semantic Indexing, Hidden Markov Models, Matplotlib/Seaborn/Pandas visualization.

What to actually learn

- Clustering: grouping unlabeled data by similarity. K-Means: iteratively assigns points to the nearest of k centroids, then recomputes centroids.
- EM algorithm and Gaussian Mixture Models: a softer version of clustering — models data as coming from a mixture of Gaussian distributions, assigning probabilities of cluster membership rather than hard labels.
- PCA (Principal Component Analysis): reduces dimensionality by projecting data onto the directions of maximum variance. This is the one classical technique with a real, demonstrable link to your RAG project (see below).
- ICA (Independent Component Analysis): separates a signal into statistically independent components — classic example is separating mixed audio sources ("cocktail party problem").
- Latent Semantic Indexing (LSI): an older technique for representing document meaning using SVD on a term-document matrix — conceptually the ancestor of modern embeddings, just linear-algebra-based instead of neural-network-based.
- Hidden Markov Models (HMMs): model sequences where the true state is hidden but produces observable outputs (classic uses: speech recognition, POS tagging).
- Data visualization: Matplotlib for basic plots, Seaborn for statistical visualizations, Pandas' built-in `.plot()` for quick exploratory plots.

Hands-on

- Run K-Means on a simple 2D dataset, visualize the clusters and centroids with Matplotlib.
- Take the actual embeddings from your RAG project's ChromaDB collection, reduce them to 2D with PCA (or t-SNE), and plot them with Seaborn — chunks from the same document/topic should visibly cluster together.

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# embeddings pulled from your Chroma collection
reduced = PCA(n_components=2).fit_transform(embeddings)
plt.scatter(reduced[:, 0], reduced[:, 1])
plt.title("RAG chunk embeddings, reduced to 2D via PCA")
plt.show()
```

Link back to RAG: This is a genuinely strong addition to your RAG project, not just a syllabus checkbox. A 2D visualization showing your document chunks clustering by topic is a great slide for your Day 6 presentation, and it directly demonstrates that your embeddings are capturing meaningful semantic structure — exactly the kind of visual proof that makes a RAG demo memorable instead of generic.

Honest note: ICA, HMMs, and LSI itself have no direct use in your RAG pipeline — sentence-transformer embeddings replace what LSI used to do, and RAG doesn't involve sequential hidden-state modeling or signal separation. Know the concepts for the syllabus, but don't try to force them into your project.

Checkpoint: K-Means clustering visualized on a toy dataset, plus a PCA visualization of your actual RAG embeddings showing topic clusters.

Day 11 — Reinforcement Learning

Syllabus topics: MDPs, Bellman Equations, Linear Quadratic Regulation (LQR), Q-Learning.

What to actually learn

- Markov Decision Process (MDP): a framework for decision-making with states, actions, transition probabilities, and rewards, where the next state depends only on the current state and action (the Markov property).
- Bellman equations: express the value of a state recursively in terms of the values of successor states — the mathematical backbone of almost all RL algorithms.
- Q-Learning: a model-free RL algorithm that learns the expected value (Q-value) of taking an action in a state, updated iteratively via the Bellman equation, without needing to know the environment's transition probabilities in advance.
- LQR (Linear Quadratic Regulation): an optimal control technique for systems with linear dynamics and quadratic cost — mostly relevant in robotics/control theory context; know the definition and that it's a classical control-theory counterpart to RL, not a deep-learning method.

Hands-on

- Implement tabular Q-Learning on a simple grid world (e.g. a 4x4 or 5x5 grid with a goal and obstacles) — this is small enough to fully understand and is the standard "hello world" of RL.

```
import numpy as np

Q = np.zeros((n_states, n_actions))
alpha, gamma, epsilon = 0.1, 0.9, 0.1

for episode in range(num_episodes):
    state = env.reset()
    done = False
    while not done:
        if np.random.rand() < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(Q[state])
        next_state, reward, done, _ = env.step(action)
        Q[state, action] += alpha * (reward + gamma * np.max(Q[next_state]) - Q[state,
action])
        state = next_state
```

Honest note: Reinforcement learning has no connection to RAG whatsoever — RAG has no reward signal, no environment, no sequential decision-making. This is purely syllabus coverage. Gymnasium (formerly OpenAI Gym) has a built-in FrozenLake environment that's perfect for this exercise if you want a ready-made grid world instead of writing one from scratch.

Checkpoint: A working tabular Q-Learning agent that solves a simple grid world, with a plot of reward improving over episodes.

Day 12 — Deep Learning Architectures

Syllabus topics: Introduction to Neural Networks, ANN, CNN, RNN/LSTM/GRU, Transfer Learning.

What to actually learn

- Artificial Neural Network (ANN): layers of neurons, each computing a weighted sum + nonlinear activation function; trained via backpropagation and gradient descent to minimize a loss function.
- Convolutional Neural Network (CNN): uses convolutional filters to detect spatial patterns (edges, textures, shapes) — the standard architecture for image data.
- Recurrent Neural Network (RNN) and its variants LSTM/GRU: process sequential data by maintaining a hidden state across time steps; LSTM/GRU add gating mechanisms to handle long-range dependencies that plain RNNs struggle with.
- Transfer learning: taking a model pretrained on a large dataset and fine-tuning it (or using it as a fixed feature extractor) on a smaller, task-specific dataset — this is exactly what you already did in RAG without necessarily naming it that way.

Hands-on

- Build a minimal ANN from scratch in NumPy (2-3 layers, manual forward pass and gradient descent) on a toy dataset — do this once so backpropagation isn't a black box.
- Train a small CNN in PyTorch or TensorFlow/Keras on MNIST or CIFAR-10 — a few epochs is enough to see it work.

Link back to RAG: This is the day that retroactively explains your RAG project's internals. The sentence-transformer model you used for embeddings on Day 3 is a transformer-based neural network (a descendant of the same neural network family as ANN/RNN, using an attention mechanism instead of recurrence or convolution). The Gemini LLM you called on Day 4 is also a transformer. You used both as pretrained black boxes via transfer learning — that's not a limitation, it's the standard, correct way to build RAG. Now you can actually explain what was inside those black boxes if asked.

Honest note: CNNs specifically (image-focused architecture) have zero role in a text-based RAG app — that connection only exists for the Day 13 image classification project, not for RAG.

Checkpoint: A from-scratch NumPy neural network trained on a toy dataset, plus a small CNN trained on MNIST/CIFAR-10.

Day 13 — OpenCV and Image Classification Project

Syllabus topics: Image Processing (OpenCV), Projects — Image Classification.

What to actually learn

- OpenCV basics: reading/writing images, color space conversion, resizing, basic filters (blur, edge detection with Canny) — the standard preprocessing toolkit for computer vision.
- Image classification pipeline: load a labeled image dataset, preprocess (resize, normalize), train or fine-tune a CNN, evaluate accuracy.

Hands-on — the syllabus's own suggested project

- Use a small labeled image dataset (e.g. CIFAR-10, or a Kaggle cats-vs-dogs subset). Preprocess with OpenCV, then fine-tune a pretrained CNN (e.g. MobileNet or ResNet via transfer learning — reusing the same transfer learning concept from Day 12) rather than training from scratch, since that's both faster and more realistic for a course-timeline project.

```
import cv2
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

img = cv2.imread("sample.jpg")
img = cv2.resize(img, (224, 224))
img = preprocess_input(img)

base_model = MobileNetV2(weights="imagenet", include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False
# add a small classification head on top, train on your labeled dataset
```

***Honest note:** This project is entirely separate from RAG — different domain (vision vs. text), different pipeline, different purpose. Present it as its own small project in your portfolio, not as part of the RAG deliverable.*

Checkpoint: An image classifier fine-tuned on a small labeled dataset, with reported accuracy on a held-out test set.

Day 14 — Object Detection Project + Full Consolidation

Syllabus topics: Projects — Object Detection. Plus: consolidation of both tracks (RAG + syllabus) for presentation.

What to actually learn

- Object detection vs. classification: classification says "what is in this image," detection says "what is in this image and where" (bounding boxes + class labels for potentially multiple objects).
- You don't need to train an object detector from scratch — that's a multi-day task on its own even for experienced practitioners. Use a pretrained model (e.g. YOLOv8 via the `ultralytics` package, or OpenCV's built-in pretrained MobileNet-SSD) and run inference on sample images/webcam.

Hands-on — the syllabus's own suggested project

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt") # pretrained, nano version – fast enough for CPU
results = model("sample.jpg")
results[0].show() # draws bounding boxes + labels
```

***Honest note:** Like Day 13, this is a standalone project with no connection to RAG. Don't try to merge it with your PDF Q&A app — they solve different problems.*

Consolidation — for the rest of Day 14

- Re-run your RAG app end to end once more to make sure nothing broke since Day 6.
- Prepare a single combined portfolio summary: RAG app (primary project, deep understanding, full pipeline) + a short section on the classical AI/ML/DL/RL/CV syllabus work (agents/search, classical ML comparison, clustering + PCA visualization tied back to RAG, Q-learning, neural net fundamentals, image classification, object detection).
- Re-read your Day 6 interview question list, and add two more you should now be ready for: "What's the difference between supervised, unsupervised, and reinforcement learning?" and "How does a transformer-based embedding model differ from classical techniques like TF-IDF or LSI?" — both are now things you've actually built small demonstrations of, not just memorized definitions.

Checkpoint: A working object detector on sample images, plus a single consolidated summary covering the RAG project and all syllabus topics, ready to present or discuss.

Overall Time Budget

- Days 1-6 (RAG): ~3-4 hrs/day, ~20-24 hours total.
- Days 7-14 (syllabus): ~5-6 hrs/day given the intensive 2-week timeline, ~40-48 hours total.
- Total: roughly 60-70 hours across 14 days. This is genuinely intensive — if a day runs long, let it spill into the next rather than skipping the hands-on exercise. Understanding beats pace on every single day here.

The One Rule That Still Matters Most

For the RAG project (Days 1-6): if you can't explain why you used ChromaDB over a keyword search, you're not done with that day.

For the syllabus material (Days 7-14): if you can't say in one sentence what a topic is used for and give one real example, you're not done with that day either — memorized definitions without a concrete example are exactly what falls apart under a follow-up question.